

in Figure 1 illustrates the overall structure of the Node.js framework.

The major blocks that the Node.js framework consists of are the interface APIs, a thin core and low lying libraries for event management, the HTTP server, security and the V8 engine. The framework also provides for scaling up the functionality as add-ons, as detailed in the next section.

Event loop

In an embedded firmware solution that has no scheduler to manage its different activities, a design involving interrupts and an infinite loop is used. The system's activities, triggered by external interrupts or internal timer interrupts, are created as events and managed in the infinite loop outside the interrupt context. The way the loop manages the events is based on the application and the needs of the embedded solution.

On similar principles, the Node.js framework uses the event loop to manage different activities of the framework. This implementation is done in the lower library *libuv* available in <http://libuv.org/>. *Libuv* is a cross-platform library aimed at providing a framework for event-driven asynchronous I/O. Figure 2 illustrates the event loop of the Node.js framework.

A detailed explanation of how the event loop is implemented within the Node.js framework is available at <https://github.com/nodejs/node/blob/master/doc/topics/event-loop-timers-and-nexttick.md>.

Non-blocking I/O

Asynchronous I/O operations using call back form the core part of the non-blocking I/O operation of the Node.js framework, as illustrated in Figure 3.

To provide a non-blocking design, the framework offloads I/O operations to the underlying operating system and is managed within the event loop. Once the operation is completed, the operating system will get back to the Node.js framework. This control back to the framework is via the call back that is passed while calling the I/O operation.

Talking to your Bluetooth device over Node.js

To scale up, Node.js extends the functionality by installing add-ons as packages. In this following example, let us explore how to interface Node.js with the Linux Bluetooth framework. The set-up is illustrated in Figure 4.

This section assumes that the reader has prior understanding of the BlueZ Linux driver and the DBus framework.

For example, the application that runs on the Node.js framework is a simple Web server that discovers Bluetooth devices in the vicinity and lists them on the browser. Let's assume that a Linux system with a BlueZ stack is running a Node.js Web server with the package *dbus-native*. Given below is the code snippet of the application that creates a

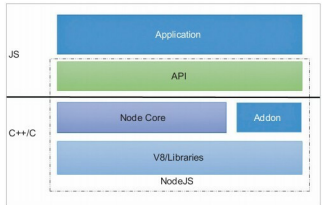


Figure 1: Blocks of the Node.js framework



Figure 2: Illustrative representation of an event loop

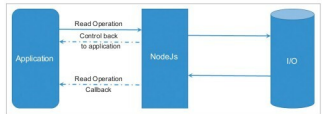


Figure 3: A representation of non-blocking I/O

Web server on port '8080' and starts discovering Bluetooth devices.

```
var dbus = require('dbus-native');
var sBus = dbus.systemBus();

var systemBus = dbus.systemBus();
var btService = systemBus.getService('org.bluez');

bt.getInterface('/org/bluez/hci0', 'org.bluez.Adapter1', function(err, Intf){
  Intf.StartDiscovery();
});
```